COURS : RASTERISATION 2D/3D

De l'intuition à l'optimisation, avec implémentation C++ from scratch

Table des matières détaillée

1. Fondamentaux : Le Framebuffer et la Géométrie Discrète
    - 1.1 Qu'est-ce qu'un pixel ?
    - 1.2 Coordonnées continues vs discrètes
    - 1.3 Le framebuffer : structure mémoire
2. Rasterisation de segments : du naïf à l'optimal
    - 2.1 Algorithme naïf : équation de droite
    - 2.2 Algorithme DDA (Digital Differential Analyzer)
    - 2.3 Algorithme de Bresenham : approche par erreur
    - 2.4 Algorithme symétrique de Bresenham
    - 2.5 Algorithme de Xiaolin Wu pour l'anti-aliasing
3. Rasterisation de cercles et ellipses
    - 3.1 Algorithme naïf : paramétrique trigonométrique
    - 3.2 Algorithme du point médian (Midpoint Circle)
    - 3.3 Algorithmes pour les ellipses
4. Rasterisation de polygones : remplissage
    - 4.1 Algorithme naïf : test de point dans polygone
    - 4.2 Scanline Polygon Fill (remplissage par balayage)
    - 4.3 Gestion des cas particuliers (sommets horizontaux)
    - 4.4 Flood Fill et Boundary Fill
5. Rasterisation 3D : triangles et interpolation
    - 5.1 Pourquoi des triangles ?
    - 5.2 Algorithme de remplissage de triangle : bounding box + barycentrique
    - 5.3 Interpolation linéaire (LERP) et ses limites
    - 5.4 Coordonnées barycentriques : théorie et calcul pratique
    - 5.5 Z-buffering : gestion de la profondeur
    - 5.6 Gouraud shading : interpolation de couleurs
    - 5.7 Texture mapping : interpolation UV
    - 5.8 Correction de perspective : pourquoi c'est nécessaire
    - 5.9 Implémentation complète avec correction de perspective
6. Optimisations avancées

TEUGUIA TADJUIDJE RODOLF SÉDÉRIS
rihen.universe@gmail.com • teuguiasederis@gmail.com
(+237) 693-761-773
CEO – RIHEN

© 2025 Rihen — Tous droits réservés

- 6.1 Rasterisation par scanline
- 6.2 Edge walking vs edge equations
- 6.3 Subpixel precision
- 6.4 Tile-based rasterization

7. Implémentation C++ complète from scratch

- 7.1 Structure du projet
- 7.2 Classes fondamentales (Vec2, Vec3, Color, Image, ZBuffer)
- 7.3 Implémentation de tous les algorithmes
- 7.4 Exemple complet : rasterizer d'un fichier OBJ

---

1. Fondamentaux : Le Framebuffer et la Géométrie Discrète

1.1 Qu'est-ce qu'un pixel ?

Un **pixel** (picture element) est la plus petite unité adressable d'une image numérique. En rasterisation, nous travaillons sur une grille discrète de pixels.

Propriétés fondamentales :

- Chaque pixel a des coordonnées entières `(x, y)`
- La convention standard : `(0, 0)` en haut à gauche, `x` croissant vers la droite, `y` croissant vers le bas
- Chaque pixel représente un petit carré (ou point) de couleur

1.2 Coordonnées continues vs discrètes

Le problème fondamental de la rasterisation : **nous voulons représenter des primitives géométriques continues (lignes, cercles, triangles) sur une grille discrète de pixels.**

```
Monde continu (mathématique)    →    Monde discret (écran)
Point (1.5, 2.7)                →    Pixel (1, 2) ou (2, 3) ?
Ligne de (0,0) à (5,3)          →    Quels pixels allumer ?
```

Cette discrétisation introduit des erreurs d'échantillonnage : **crénelage (aliasing)**.

1.3 Le framebuffer : structure mémoire

Le framebuffer est un tableau linéaire en mémoire. Organisation typique :

```
Adresse mémoire = (y * width + x) * bytes_per_pixel
```

Pour une image RGB 24 bits (3 octets par pixel) :

© 2025 Rihen — Tous droits réservés

rihen.universe@gmail.com • teuguiasederis@gmail.com
(+237) 693-761-773

```
// Accès à un pixel
int index = y * width + x;
unsigned char red   = framebuffer[index * 3 + 0];
unsigned char green = framebuffer[index * 3 + 1];
unsigned char blue  = framebuffer[index * 3 + 2];
```

Pourquoi cette organisation ? Parce que la mémoire est linéaire, et parcourir les pixels ligne par ligne (scanline) est optimal pour la localité des données (cache CPU).

---

2. Rasterisation de segments : du naïf à l'optimal

2.1 Algorithme naïf : équation de droite

Approche intuitive : Une droite est définie par l'équation `y = m*x + b`, où `m` est la pente et `b` l'ordonnée à l'origine.

Algorithme naïf :

```
void drawLineNaive(float x0, float y0, float x1, float y1, Color color) {
    float m = (y1 - y0) / (x1 - x0);  // Calcul de la pente
    float b = y0 - m * x0;            // Calcul de l'ordonnée à l'origine

    for (int x = round(x0); x <= round(x1); x++) {
        float y = m * x + b;          // Équation de la droite
        int y_pixel = round(y);       // Arrondi au pixel le plus proche
        setPixel(x, y_pixel, color);
    }
}
```

Pourquoi cet algorithme pose problème ?

1. **Divisions :** `m = dy/dx` nécessite une division (coûteuse)
2. **Flottants :** Utilise des nombres flottants (plus lents que les entiers)
3. **Accumulation d'erreur :** Les arrondis successifs peuvent décaler la ligne
4. **Cas vertical :** `dx = 0` → division par zéro !
5. **Pente > 1 :** La ligne devient discontinue (trous)

Exemple concret : Pour une ligne de (0,0) à (2,10), avec une pente de 5 :

- x=0 → y=0 → pixel (0,0)
- x=1 → y=5 → pixel (1,5)
- x=2 → y=10 → pixel (2,10)
- Les pixels (0,1), (0,2), (0,3), (0,4) sont manquants ! La ligne est discontinue.

Pourquoi ce problème ? Parce que nous incrémentons x de 1, mais y varie plus vite. Il faut adapter l'incrémentation selon la pente.

© 2025 Rihen — Tous droits réservés

2.2 Algorithme DDA (Digital Differential Analyzer)

Le DDA améliore l'approche naïve en choisissant le pas d'incrémentation selon la pente.

Principe : On parcourt la ligne en incrémentant la coordonnée qui varie le plus (la dominante), et on calcule l'autre par incréments proportionnels.

Pourquoi "DDA" ? C'est une méthode numérique pour intégrer des équations différentielles : `dx/dt = Δx`, `dy/dt = Δy`.

Algorithme DDA :

```
void drawLineDDA(float x0, float y0, float x1, float y1, Color color) {
    float dx = x1 - x0;
    float dy = y1 - y0;

    // Déterminer le nombre de pas
    int steps = max(abs(dx), abs(dy));

    // Calculer les incréments
    float xInc = dx / steps;
    float yInc = dy / steps;

    float x = x0;
    float y = y0;

    for (int i = 0; i <= steps; i++) {
        setPixel(round(x), round(y), color);
        x += xInc;
        y += yInc;
    }
}
```

Avantages du DDA :

- Gère tous les cas (pentes, verticales, horizontales)
- Pas de division par zéro (steps est toujours ≥ 0)
- Plus robuste que l'approche naïve

Inconvénients :

- Utilise toujours des flottants
- Deux divisions (`dx/steps` et `dy/steps`) sont coûteuses
- Arrondis à chaque itération

Exemple détaillé : Ligne de (0,0) à (2,10)

- dx = 2, dy = 10, steps = 10
- xInc = 0.2, yInc = 1.0
- Itérations :

© 2025 Rihen — Tous droits réservés

- i=0 : x=0, y=0 → pixel (0,0)
- i=1 : x=0.2, y=1 → pixel (0,1)
- i=2 : x=0.4, y=2 → pixel (0,2)
- ...
- i=10 : x=2, y=10 → pixel (2,10)

Maintenant la ligne est continue !

2.3 Algorithme de Bresenham : approche par erreur

Bresenham a révolutionné le tracé de lignes en 1965 en utilisant uniquement des entiers et des additions/soustractions.

Philosophie : Au lieu de calculer explicitement y = m*x, on maintient une **variable d'erreur** qui mesure l'écart entre la position idéale et le pixel actuel. On décide d'incrémenter y lorsque l'erreur dépasse un seuil.

Pourquoi cette approche ? Parce que sur un écran, les pixels sont entiers. Le choix est binaire : soit on reste sur la même ligne horizontale, soit on monte (ou descend) d'un pixel.

Explication pas à pas :

1. **Cas simple : pente entre 0 et 1** (dx > dy)
   - On incrémente x de 1 à chaque étape
   - La position réelle de la ligne est : `y = (dy/dx)*x + b`
   - Entre deux pixels horizontaux, on a deux choix :
     - Rester sur le même y : erreur = y_idéal - y_pixel_actuel
     - Monter d'un pixel : erreur = (y_idéal - (y_pixel_actuel + 1))
2. **Variable de décision :**
   Soit `d = 2*dy*x - 2*dx*y + c` (forme entière après manipulation)
   On peut mettre à jour d incrémentalement :
   - Si d < 0 : on reste sur le même y, d += 2*dy
   - Si d ≥ 0 : on incrémente y, d += 2*(dy - dx)

Algorithme de Bresenham (cas 0 < m < 1) :

```
void drawLineBresenham(int x0, int y0, int x1, int y1, Color color) {
    int dx = abs(x1 - x0);
    int dy = abs(y1 - y0);
    int sx = (x0 < x1) ? 1 : -1;  // Direction en x
    int sy = (y0 < y1) ? 1 : -1;  // Direction en y

    // Variable de décision
    int err = dx - dy;

    while (true) {
        setPixel(x0, y0, color);
```


© 2025 Rihen — Tous droits réservés

```
        if (x0 == x1 && y0 == y1) break;

        int e2 = 2 * err;

        // Décision pour x
        if (e2 > -dy) {
            err -= dy;
            x0 += sx;
        }

        // Décision pour y
        if (e2 < dx) {
            err += dx;
            y0 += sy;
        }
    }
}
```

Pourquoi e2 = 2*err ? Pour éviter les divisions, on multiplie par 2. La comparaison `e2 > -dy` est équivalente à `err > -dy/2` mais sans division.

Pourquoi err = dx - dy initialement ? C'est le point médian entre le premier et le deuxième pixel. Cette valeur initiale garantit que l'algorithme est symétrique.

Déroulé détaillé pour la ligne (0,0) → (5,3) :

- dx = 5, dy = 3, sx = 1, sy = 1, err = 2
- Pixel (0,0)
- e2 = 4
  - e2 > -dy ? 4 > -3 → oui → err = 2-3 = -1, x=1
  - e2 < dx ? 4 < 5 → oui → err = -1+5 = 4, y=1
- Pixel (1,1)
- e2 = 8
  - 8 > -3 → oui → err = 4-3=1, x=2
  - 8 < 5 → non
- Pixel (2,1)
- e2 = 2
  - 2 > -3 → oui → err = 1-3=-2, x=3
  - 2 < 5 → oui → err = -2+5=3, y=2
- Pixel (3,2)
- ... et ainsi de suite

Résultat : Pixels (0,0), (1,1), (2,1), (3,2), (4,2), (5,3)

2.4 Algorithme symétrique de Bresenham

La version précédente fonctionne pour toutes les pentes grâce à la symétrie des décisions. Les tests `e2 > -dy` et `e2 < dx` fonctionnent quelle que soit la pente.

© 2025 Rihen — Tous droits réservés

Pourquoi ces deux tests ?

- `e2 > -dy` : décide d'incrémenter x (toujours nécessaire)
- `e2 < dx` : décide d'incrémenter y (selon la pente)

2.5 Algorithme de Xiaolin Wu pour l'anti-aliasing

Bresenham donne des lignes "crénelées" (effet d'escalier). Xiaolin Wu (1991) propose un algorithme avec **anti-aliasing** en utilisant des niveaux de gris.

Principe : Pour chaque pixel traversé par la ligne, on calcule sa distance à la ligne idéale et on attribue une intensité proportionnelle.

Algorithme simplifié :

```
void drawLineWu(float x0, float y0, float x1, float y1, Color color) {
    bool steep = abs(y1 - y0) > abs(x1 - x0);
    if (steep) {
        swap(x0, y0);
        swap(x1, y1);
    }
    if (x0 > x1) {
        swap(x0, x1);
        swap(y0, y1);
    }

    float dx = x1 - x0;
    float dy = y1 - y0;
    float gradient = dy / dx;

    // Premier pixel
    float y = y0 + gradient * (round(x0) - x0);
    for (int x = round(x0); x <= round(x1); x++) {
        float alpha = 1.0f - abs(y - round(y));
        if (steep) {
            setPixel(round(y), x, color * alpha);
            setPixel(round(y) + 1, x, color * (1 - alpha));
        } else {
            setPixel(x, round(y), color * alpha);
            setPixel(x, round(y) + 1, color * (1 - alpha));
        }
        y += gradient;
    }
}
```

Pourquoi ça marche ? En répartissant l'intensité entre deux pixels voisins, l'œil perçoit une ligne plus lisse, comme s'il y avait un sous-pixel.

---

3. Rasterisation de cercles et ellipses

© 2025 Rihen — Tous droits réservés

3.1 Algorithme naïf : paramétrique trigonométrique

L'approche intuitive : utiliser l'équation paramétrique du cercle.

Équation : `x = xc + r * cos(θ)`, `y = yc + r * sin(θ)`

Algorithme naïf :

```
void drawCircleNaive(int xc, int yc, int r, Color color) {
    for (float theta = 0; theta < 2 * M_PI; theta += 0.01) {
        int x = xc + r * cos(theta);
        int y = yc + r * sin(theta);
        setPixel(x, y, color);
    }
}
```

Pourquoi cet algorithme est problématique ?

1. **Choix du pas :** Trop grand → trous, trop petit → sur-échantillonnage
2. **Calculs trigonométriques :** `cos` et `sin` sont très lents
3. **Symétrie non exploitée :** Le cercle a 8 symétries
4. **Flottants :** Accumulation d'erreurs

La symétrie du cercle : Un cercle a 8 octants symétriques. Si on calcule les points pour un octant (x de 0 à r/√2), on peut générer les 7 autres par symétrie :

- (x, y) → (y, x)
- (x, y) → (-x, y)
- (x, y) → (x, -y)
- (x, y) → (-x, -y)
- (x, y) → (y, x)
- etc.

3.2 Algorithme du point médian (Midpoint Circle)

Bresenham a aussi proposé un algorithme pour les cercles, basé sur le même principe que pour les lignes.

Principe : On trace le cercle dans le premier octant (x de 0 à r/√2). À chaque étape, on a deux choix pour le pixel suivant :

- Est (E) : (x+1, y)
- Sud-Est (SE) : (x+1, y-1)

On évalue la fonction du cercle au point médian entre E et SE :

```
f(x, y) = x² + y² - r²
f < 0 → point à l'intérieur du cercle
```


© 2025 Rihen — Tous droits réservés

```
f = 0 → point sur le cercle
f > 0 → point à l'extérieur
```

Variable de décision : `d = f(x+1, y-0.5)`

Mise à jour :

- Si d < 0 : choisir E, `d += 2*x + 3`
- Si d ≥ 0 : choisir SE, `d += 2*(x - y) + 5`

Algorithme complet :

```
void drawCircleMidpoint(int xc, int yc, int r, Color color) {
    int x = 0;
    int y = r;
    int d = 1 - r;  // Variable de décision initiale (pour r entier)

    // Fonction pour dessiner les 8 points symétriques
    auto drawSymmetry = [&](int xc, int yc, int x, int y) {
        setPixel(xc + x, yc + y, color);
        setPixel(xc - x, yc + y, color);
        setPixel(xc + x, yc - y, color);
        setPixel(xc - x, yc - y, color);
        setPixel(xc + y, yc + x, color);
        setPixel(xc - y, yc + x, color);
        setPixel(xc + y, yc - x, color);
        setPixel(xc - y, yc - x, color);
    };

    drawSymmetry(xc, yc, x, y);

    while (x < y) {
        x++;
        if (d < 0) {
            d += 2*x + 1;
        } else {
            y--;
            d += 2*(x - y) + 1;
        }
        drawSymmetry(xc, yc, x, y);
    }
}
```

Pourquoi d = 1 - r initialement ? C'est la valeur de f(1, r-0.5) = 1 + (r^2 - r + 0.25) - r^2 = 1.25 - r. En prenant la partie entière, on obtient 1 - r.

Pourquoi la boucle s'arrête quand x >= y ? Parce qu'on a parcouru tout le premier octant (45 degrés). Après ce point, on commence à répéter les pixels.

3.3 Algorithmes pour les ellipses

L'ellipse est plus complexe car elle n'a que 4 symétries (pas 8 comme le cercle).

Équation : $(x - xc)^2/a^2 + (y - yc)^2/b^2 = 1$

On utilise un algorithme à deux régions :

1. **Région 1 :** pente < 1 (dominante x), on incrémente x
2. **Région 2 :** pente > 1 (dominante y), on incrémente y

Principe : On calcule la variable de décision pour l'ellipse et on change de région quand la pente atteint -1.

---

4. Rasterisation de polygones : remplissage

4.1 Algorithme naïf : test de point dans polygone

Approche intuitive : Pour chaque pixel de l'image, tester s'il est à l'intérieur du polygone.

Test de point dans polygone :

- **Méthode du ray casting** : Tracer une demi-droite horizontale vers la droite et compter les intersections avec les arêtes du polygone.
- Si le nombre d'intersections est impair → point à l'intérieur

```
bool isPointInPolygon(int x, int y, const vector<Vec2>& vertices) {
    bool inside = false;
    int n = vertices.size();

    for (int i = 0, j = n-1; i < n; j = i++) {
        const Vec2& vi = vertices[i];
        const Vec2& vj = vertices[j];

        // Vérifier si l'arête intersecte la demi-droite horizontale
        bool intersect = ((vi.y > y) != (vj.y > y)) &&
                         (x < (vj.x - vi.x) * (y - vi.y) / (vj.y - vi.y) +
vi.x);

        if (intersect) inside = !inside;
    }
    return inside;
}
```

Pourquoi cet algorithme est-il inefficace ?

- Complexité : O(width × height × n), où n est le nombre de sommets
- Beaucoup de calculs inutiles : on teste tous les pixels même loin du polygone
- Pas de cohérence entre pixels adjacents

© 2025 Rihen — Tous droits réservés

4.2 Scanline Polygon Fill (remplissage par balayage)

C'est l'algorithme standard pour remplir des polygones dans les années 80-90.

Principe : On parcourt l'image ligne par ligne (scanline). Pour chaque ligne y, on calcule toutes les intersections avec les arêtes du polygone, on les trie par x croissant, et on remplit entre les paires d'intersections.

Structure de données :

- **Edge Table (ET)** : Pour chaque y (de ymin à ymax), liste des arêtes commençant à ce y
- **Active Edge Table (AET)** : Liste des arêtes qui intersectent la scanline courante

Chaque arête est représentée par :

- `ymax` : valeur y maximale de l'arête
- `x` : coordonnée x du point d'intersection avec la scanline courante
- `dx` : incrément de x par scanline (1/pente)

Algorithme détaillé :

```
struct Edge {
    int ymax;
    float x;
    float dx;
};

void fillPolygonScanline(const vector<Vec2>& vertices, Color color, Image& img)
{
    // 1. Trouver ymin et ymax du polygone
    int ymin = img.getHeight(), ymax = 0;
    for (const auto& v : vertices) {
        ymin = min(ymin, (int)v.y);
        ymax = max(ymax, (int)v.y);
    }

    // 2. Construire l'Edge Table
    vector<vector<Edge>> edgeTable(ymax - ymin + 1);

    int n = vertices.size();
    for (int i = 0; i < n; i++) {
        const Vec2& v1 = vertices[i];
        const Vec2& v2 = vertices[(i+1) % n];

        // Ignorer les arêtes horizontales
        if (v1.y == v2.y) continue;

        // S'assurer que v1 est le point le plus bas
        const Vec2& bottom = (v1.y < v2.y) ? v1 : v2;
        const Vec2& top = (v1.y < v2.y) ? v2 : v1;
```

```
        float dx = (top.x - bottom.x) / (top.y - bottom.y);

        Edge edge;
        edge.ymax = (int)top.y;
        edge.x = bottom.x;
        edge.dx = dx;

        edgeTable[(int)bottom.y - ymin].push_back(edge);
    }

    // 3. Scanline
    vector<Edge> activeEdges;

    for (int y = ymin; y <= ymax; y++) {
        // Ajouter les arêtes commençant à cette scanline
        for (const auto& edge : edgeTable[y - ymin]) {
            activeEdges.push_back(edge);
        }

        // Trier les arêtes actives par x croissant
        sort(activeEdges.begin(), activeEdges.end(),
             [](const Edge& a, const Edge& b) { return a.x < b.x; });

        // Remplir entre paires d'intersections
        for (size_t i = 0; i < activeEdges.size(); i += 2) {
            if (i+1 >= activeEdges.size()) break;

            int x1 = (int)ceil(activeEdges[i].x);
            int x2 = (int)floor(activeEdges[i+1].x);

            for (int x = x1; x <= x2; x++) {
                img.setPixel(x, y, color);
            }
        }

        // Mettre à jour les arêtes actives pour la prochaine scanline
        for (auto it = activeEdges.begin(); it != activeEdges.end();) {
            it->x += it->dx;
            if (it->ymax == y) {
                it = activeEdges.erase(it);
            } else {
                ++it;
            }
        }
    }
}
```

Pourquoi cet algorithme est efficace ?

- Complexité : O(n × nombre de lignes), beaucoup mieux que le test de point
- Utilise la cohérence spatiale : les intersections changent peu entre lignes adjacentes

© 2025 Rihen — Tous droits réservés

- Ne traite que les pixels à l'intérieur du polygone

4.3 Gestion des cas particuliers

Problème des sommets horizontaux : Quand un sommet est exactement sur une scanline, on peut compter l'intersection deux fois (si on prend les deux arêtes).

Solution : Règle du "top-left rule" : on prend en compte l'arête si le sommet est le point le plus bas de l'arête, mais pas si c'est le point le plus haut.

Dans notre implémentation : On ignore les arêtes horizontales (v1.y == v2.y) et on stocke les arêtes avec le point bas (bottom) comme point de départ.

4.4 Flood Fill et Boundary Fill

Flood Fill : Remplit une région connectée à partir d'un point de départ.

Algorithme récursif (4-connexe) :

```
void floodFill(int x, int y, Color fillColor, Color targetColor, Image& img) {
    if (x < 0 || x >= img.getWidth() || y < 0 || y >= img.getHeight()) return;
    if (img.getPixel(x, y) != targetColor) return;

    img.setPixel(x, y, fillColor);

    floodFill(x+1, y, fillColor, targetColor, img);
    floodFill(x-1, y, fillColor, targetColor, img);
    floodFill(x, y+1, fillColor, targetColor, img);
    floodFill(x, y-1, fillColor, targetColor, img);
}
```

Pourquoi éviter la récursion ? La profondeur de récursion peut être énorme (jusqu'à la taille de l'image) et faire déborder la pile.

Version itérative avec pile :

```
void floodFillIterative(int x, int y, Color fillColor, Color targetColor,
Image& img) {
    stack<pair<int,int>> pixels;
    pixels.push({x, y});

    while (!pixels.empty()) {
        auto [px, py] = pixels.top();
        pixels.pop();

        if (px < 0 || px >= img.getWidth() || py < 0 || py >= img.getHeight())
continue;
        if (img.getPixel(px, py) != targetColor) continue;
```


© 2025 Rihen — Tous droits réservés

```
        img.setPixel(px, py, fillColor);

        pixels.push({px+1, py});
        pixels.push({px-1, py});
        pixels.push({px, py+1});
        pixels.push({px, py-1});
    }
}
```

---

5. Rasterisation 3D : triangles et interpolation

5.1 Pourquoi des triangles ?

Le triangle est la primitive 3D fondamentale car :

1. **Coplanarité :** Trois points définissent toujours un plan (pas de distortion)
2. **Simplicité :** L'interpolation barycentrique est simple
3. **Rasterisation facile :** Les algorithmes sont bien établis
4. **Toute surface peut être tessellée en triangles**

5.2 Algorithme de remplissage de triangle : bounding box + barycentrique

Approche naïve : Pour chaque triangle, déterminer son bounding box (rectangle englobant), puis tester chaque pixel du bounding box pour savoir s'il est à l'intérieur du triangle.

```
void drawTriangleNaive(const Vec2f& v0, const Vec2f& v1, const Vec2f& v2,
                       Color color, Image& img) {
    // Calculer le bounding box
    int minX = min({v0.x, v1.x, v2.x});
    int maxX = max({v0.x, v1.x, v2.x});
    int minY = min({v0.y, v1.y, v2.y});
    int maxY = max({v0.y, v1.y, v2.y});

    // Pour chaque pixel dans le bounding box
    for (int y = minY; y <= maxY; y++) {
        for (int x = minX; x <= maxX; x++) {
            Vec2f p(x + 0.5f, y + 0.5f);  // Centre du pixel

            if (isPointInTriangle(p, v0, v1, v2)) {
                img.setPixel(x, y, color);
            }
        }
    }
}
```

Pourquoi le décalage de 0.5 ? Le pixel `(x, y)` représente un carré allant de `(x, y)` à `(x+1, y+1)`. Le centre du pixel est `(x+0.5, y+0.5)`. C'est le point de référence standard pour les tests de couverture.

© 2025 Rihen — Tous droits réservés

5.3 Interpolation linéaire (LERP) et ses limites

LERP (Linear Interpolation) :

```
value = a * (1 - t) + b * t
```

Dans le contexte d'un triangle, on veut interpoler des attributs (couleur, UV, etc.) entre les sommets.

Problème : Dans l'espace écran (2D), les attributs ne varient pas linéairement à cause de la projection perspective.

Pourquoi ? Quand un triangle est projeté sur l'écran, les distances entre pixels ne correspondent pas aux distances dans l'espace 3D. Un objet s'éloignant semble plus petit, mais les attributs attachés à sa surface doivent "suivre" cette compression.

5.4 Coordonnées barycentriques : théorie et calcul pratique

Définition mathématique : Pour un point P dans le plan du triangle ABC, il existe trois scalaires (α, β, γ) tels que :

```
P = αA + βB + γC
α + β + γ = 1
```

Interprétation : α, β, γ sont les aires relatives des triangles PBC, APC, ABP par rapport à l'aire totale.

Calcul pratique :

```
struct Barycentric {
    float alpha, beta, gamma;
};

Barycentric computeBarycentric(const Vec2f& p, const Vec2f& a,
                               const Vec2f& b, const Vec2f& c) {
    // Vecteurs
    Vec2f v0 = b - a;
    Vec2f v1 = c - a;
    Vec2f v2 = p - a;

    // Produits scalaires
    float dot00 = v0.dot(v0);
    float dot01 = v0.dot(v1);
    float dot02 = v0.dot(v2);
    float dot11 = v1.dot(v1);
    float dot12 = v1.dot(v2);

    // Inverse du déterminant
    float invDenom = 1.0f / (dot00 * dot11 - dot01 * dot01);
```


© 2025 Rihen — Tous droits réservés

```
    // Coordonnées barycentriques
    float beta = (dot11 * dot02 - dot01 * dot12) * invDenom;
    float gamma = (dot00 * dot12 - dot01 * dot02) * invDenom;
    float alpha = 1.0f - beta - gamma;

    return {alpha, beta, gamma};
}
```

Pourquoi cette formule ? C'est la solution du système linéaire :

```
β * (B-A) + γ * (C-A) = P-A
```

En projetant sur B-A et C-A, on obtient un système 2×2 résolu avec la formule de Cramer.

5.5 Z-buffering : gestion de la profondeur

Le Z-buffer résout le problème d'occlusion : quand plusieurs triangles couvrent le même pixel, on garde celui qui est le plus proche de la caméra.

Structure : Un tableau de float de la même taille que l'image, initialisé à +∞ (ou 1.0 pour une profondeur normalisée).

Algorithme :

```
class ZBuffer {
    int width, height;
    vector<float> depths;

public:
    ZBuffer(int w, int h) : width(w), height(h), depths(w * h, INFINITY) {}

    bool test(int x, int y, float z) {
        int idx = y * width + x;
        if (z < depths[idx]) {
            depths[idx] = z;
            return true;  // Pixel visible
        }
        return false;     // Pixel caché
    }
};

// Dans le rasterizer
if (zbuffer.test(x, y, interpolatedZ)) {
    img.setPixel(x, y, color);
}
```


© 2025 Rihen — Tous droits réservés

Pourquoi comparer avec < et non > ? On utilise une caméra regardant dans la direction -Z (standard OpenGL), donc les valeurs z sont négatives. Plus z est grand (moins négatif), plus l'objet est proche. En pratique, on normalise entre 0 (near) et 1 (far).

5.6 Gouraud shading : interpolation de couleurs

Principe : Calculer la couleur aux sommets (via un modèle d'éclairage), puis interpoler ces couleurs sur le triangle.

```
Color shadeVertex(const Vec3f& position, const Vec3f& normal,
                  const Light& light, const Material& material) {
    // Éclairage de Lambert (simple)
    Vec3f L = (light.position - position).normalize();
    float diffuse = max(0.0f, normal.dot(L));

    return material.color * (material.ambient + diffuse * light.intensity);
}

// Dans le rasterizer
for each pixel:
    auto [alpha, beta, gamma] = computeBarycentric(p, v0, v1, v2);

    Color finalColor = c0 * alpha + c1 * beta + c2 * gamma;
```

Limite : Gouraud ne capture pas bien les reflets spéculaires (highlights) car l'interpolation peut les "lisser" excessivement.

5.7 Texture mapping : interpolation UV

Coordonnées UV : Chaque sommet du triangle a des coordonnées de texture (u, v) entre 0 et 1.

Interpolation naïve (linéaire en écran) :

```
float u = u0 * alpha + u1 * beta + u2 * gamma;
float v = v0 * alpha + v1 * beta + v2 * gamma;
Color texColor = texture.sample(u, v);
```

Pourquoi cette approche est fausse ? Elle ignore la correction de perspective. Le résultat est une texture qui semble "se plier" ou "s'étirer" de façon incorrecte.

5.8 Correction de perspective : pourquoi c'est nécessaire

Analyse mathématique :

En 3D, la projection perspective s'écrit :

© 2025 Rihen — Tous droits réservés

```
x_screen = X / Z
y_screen = Y / Z
```

Où (X, Y, Z) sont les coordonnées dans l'espace caméra.

Pour un attribut A (couleur, u, v, etc.) qui varie linéairement en 3D :

```
A = A0 * α + A1 * β + A2 * γ
```

Mais dans l'espace écran, les coordonnées barycentriques ne sont pas linéaires. En fait, on doit interpoler A/Z et 1/Z :

```
A_screen = (A0/Z0 * α' + A1/Z1 * β' + A2/Z2 * γ') / (1/Z0 * α' + 1/Z1 * β' +
1/Z2 * γ')
```

Où α', β', γ' sont les coordonnées barycentriques **dans l'espace écran** (2D).

Implémentation correcte :

```
// Interpoler 1/Z
float invZ = invZ0 * alpha + invZ1 * beta + invZ2 * gamma;

// Interpoler U/Z et V/Z
float uOverZ = (u0 * invZ0) * alpha + (u1 * invZ1) * beta + (u2 * invZ2) *
gamma;
float vOverZ = (v0 * invZ0) * alpha + (v1 * invZ1) * beta + (v2 * invZ2) *
gamma;

// Reconstruire U et V
float u = uOverZ / invZ;
float v = vOverZ / invZ;
```

Pourquoi ça marche ? Cette formule maintient les rapports corrects le long des lignes de fuite de la perspective.

5.9 Implémentation complète avec correction de perspective

Voici un rasterizer de triangle complet avec toutes les fonctionnalités :

```
struct Vertex {
    Vec4f position;   // Position en espace écran (x, y, z, w)
    Vec3f worldPos;   // Position dans le monde pour l'éclairage
    Vec3f normal;     // Normale pour l'éclairage
```


© 2025 Rihen — Tous droits réservés

```
    Vec2f texCoord;   // Coordonnées UV
    Color color;      // Couleur (si pas de texture)
};

void rasterizeTriangle(const Vertex& v0, const Vertex& v1, const Vertex& v2,
                       const Texture* texture, const Light& light,
                       Image& img, ZBuffer& zbuffer) {
    // Convertir en 2D pour la rasterisation
    Vec2f v0_2d(v0.position.x, v0.position.y);
    Vec2f v1_2d(v1.position.x, v1.position.y);
    Vec2f v2_2d(v2.position.x, v2.position.y);

    // Calculer le bounding box
    int minX = max(0, (int)min({v0_2d.x, v1_2d.x, v2_2d.x}));
    int maxX = min(img.getWidth() - 1, (int)max({v0_2d.x, v1_2d.x, v2_2d.x}));
    int minY = max(0, (int)min({v0_2d.y, v1_2d.y, v2_2d.y}));
    int maxY = min(img.getHeight() - 1, (int)max({v0_2d.y, v1_2d.y, v2_2d.y}));

    // Pré-calculer les inverses des Z
    float invZ0 = 1.0f / v0.position.z;
    float invZ1 = 1.0f / v1.position.z;
    float invZ2 = 1.0f / v2.position.z;

    // Pour l'interpolation des couleurs (si pas de texture)
    bool hasTexture = (texture != nullptr);

    for (int y = minY; y <= maxY; y++) {
        for (int x = minX; x <= maxX; x++) {
            Vec2f p(x + 0.5f, y + 0.5f);

            // Calcul des coordonnées barycentriques
            auto [alpha, beta, gamma] = computeBarycentric(p, v0_2d, v1_2d,
v2_2d);

            // Vérifier si le point est dans le triangle
            if (alpha >= 0 && beta >= 0 && gamma >= 0) {
                // Interpolation de la profondeur avec correction de
perspective
                float invZ = alpha * invZ0 + beta * invZ1 + gamma * invZ2;
                float z = 1.0f / invZ;

                // Test Z-buffer
                if (zbuffer.test(x, y, z)) {
                    Color finalColor;

                    if (hasTexture) {
                        // Interpolation UV avec correction de perspective
                        float uOverZ = alpha * v0.texCoord.x * invZ0 +
                                       beta  * v1.texCoord.x * invZ1 +
                                       gamma * v2.texCoord.x * invZ2;
                        float vOverZ = alpha * v0.texCoord.y * invZ0 +
                                       beta  * v1.texCoord.y * invZ1 +
                                       gamma * v2.texCoord.y * invZ2;
```

```
                        float u = uOverZ / invZ;
                        float v = vOverZ / invZ;

                        finalColor = texture->sample(u, v);
                    } else {
                        // Interpolation des couleurs (Gouraud)
                        finalColor = v0.color * alpha + v1.color * beta +
v2.color * gamma;
                    }

                    // Ajouter l'éclairage si nécessaire
                    // (Dans un vrai shader, on interpolerait normales et
positions)

                    img.setPixel(x, y, finalColor);
                }
            }
        }
    }
}
```

---

6. Optimisations avancées

6.1 Rasterisation par scanline

Au lieu de parcourir toute la bounding box, on parcourt ligne par ligne et on calcule les intersections avec les arêtes du triangle.

```
void rasterizeTriangleScanline(const Vertex& v0, const Vertex& v1, const
Vertex& v2, ...) {
    // Trier les sommets par y
    if (v0_2d.y > v1_2d.y) swap(v0, v1);
    if (v0_2d.y > v2_2d.y) swap(v0, v2);
    if (v1_2d.y > v2_2d.y) swap(v1, v2);

    int yStart = v0_2d.y;
    int yEnd = v2_2d.y;

    for (int y = yStart; y <= yEnd; y++) {
        // Calculer les intersections avec les arêtes gauches et droites
        float xLeft, xRight;

        if (y < v1_2d.y) {
            // Partie supérieure (v0 -> v1 et v0 -> v2)
            xLeft = interpolateX(v0_2d, v1_2d, y);
            xRight = interpolateX(v0_2d, v2_2d, y);
        } else {
            // Partie inférieure (v1 -> v2 et v0 -> v2)
```


© 2025 Rihen — Tous droits réservés

```
            xLeft = interpolateX(v1_2d, v2_2d, y);
            xRight = interpolateX(v0_2d, v2_2d, y);
        }

        // S'assurer que xLeft <= xRight
        if (xLeft > xRight) swap(xLeft, xRight);

        // Remplir la scanline
        for (int x = ceil(xLeft); x <= floor(xRight); x++) {
            // Interpoler les attributs pour ce pixel
            // ...
        }
    }
}
```

6.2 Edge walking vs edge equations

- **Edge walking** : Suit les arêtes et remplit entre elles (scanline)
- **Edge equations** : Teste tous les pixels de la bounding box (notre approche)

Edge walking est plus efficace pour les petits triangles, edge equations est plus simple à implémenter et mieux vectorisable.

6.3 Subpixel precision

Pour éviter les artefacts aux bords des triangles, on utilise des coordonnées fixes avec plus de précision (16.16 fixed point). Cela permet de résoudre correctement les cas où deux triangles partagent une arête.

6.4 Tile-based rasterization

On divise l'écran en tuiles (tiles) de 16×16 ou 32×32 pixels. Pour chaque tuile, on teste rapidement si le triangle la couvre, et on ne rasterise que les tuiles intersectées. C'est la base des GPU modernes.

---

7. Implémentation C++ complète from scratch

7.1 Structure du projet

```
rasterizer/
├── src/
│   ├── main.cpp
│   ├── image.cpp
│   ├── zbuffer.cpp
│   ├── rasterizer.cpp
│   ├── texture.cpp
│   └── math/
│       ├── vec2.cpp
│       ├── vec3.cpp
│       ├── vec4.cpp
│       └── matrix.cpp
```


© 2025 Rihen — Tous droits réservés

```
├── include/
│   ├── image.hpp
│   ├── zbuffer.hpp
│   ├── rasterizer.hpp
│   └── ...
└── models/
    └── cube.obj
```

7.2 Classes fondamentales

```
// vec2.hpp
struct Vec2f {
    float x, y;
    Vec2f() : x(0), y(0) {}
    Vec2f(float x, float y) : x(x), y(y) {}

    Vec2f operator+(const Vec2f& v) const { return Vec2f(x+v.x, y+v.y); }
    Vec2f operator-(const Vec2f& v) const { return Vec2f(x-v.x, y-v.y); }
    Vec2f operator*(float s) const { return Vec2f(x*s, y*s); }
    float dot(const Vec2f& v) const { return x*v.x + y*v.y; }
    float length() const { return sqrt(x*x + y*y); }
};

// vec3.hpp
struct Vec3f {
    float x, y, z;
    Vec3f() : x(0), y(0), z(0) {}
    Vec3f(float x, float y, float z) : x(x), y(y), z(z) {}

    Vec3f operator+(const Vec3f& v) const { return Vec3f(x+v.x, y+v.y, z+v.z);
}
    Vec3f operator-(const Vec3f& v) const { return Vec3f(x-v.x, y-v.y, z-v.z);
}
    Vec3f operator*(float s) const { return Vec3f(x*s, y*s, z*s); }
    Vec3f cross(const Vec3f& v) const {
        return Vec3f(y*v.z - z*v.y, z*v.x - x*v.z, x*v.y - y*v.x);
    }
    float dot(const Vec3f& v) const { return x*v.x + y*v.y + z*v.z; }
    Vec3f normalize() const {
        float len = length();
        return Vec3f(x/len, y/len, z/len);
    }
    float length() const { return sqrt(x*x + y*y + z*z); }
};

// color.hpp
struct Color {
    unsigned char r, g, b;
    Color() : r(0), g(0), b(0) {}
    Color(unsigned char r, unsigned char g, unsigned char b) : r(r), g(g), b(b)
```


© 2025 Rihen — Tous droits réservés

```
{}

    Color operator*(float s) const {
        return Color(r*s, g*s, b*s);
    }

    Color operator+(const Color& c) const {
        return Color(r+c.r, g+c.g, b+c.b);
    }
};

// image.hpp
class Image {
    int width, height;
    vector<Color> pixels;

public:
    Image(int w, int h) : width(w), height(h), pixels(w*h) {}

    void setPixel(int x, int y, const Color& c) {
        if (x >= 0 && x < width && y >= 0 && y < height)
            pixels[y*width + x] = c;
    }

    Color getPixel(int x, int y) const {
        if (x >= 0 && x < width && y >= 0 && y < height)
            return pixels[y*width + x];
        return Color();
    }

    void clear(const Color& c) {
        fill(pixels.begin(), pixels.end(), c);
    }

    void savePPM(const string& filename) const {
        ofstream file(filename);
        file << "P3\n" << width << " " << height << "\n255\n";
        for (const auto& c : pixels) {
            file << (int)c.r << " " << (int)c.g << " " << (int)c.b << "\n";
        }
    }
};
```

7.3 Implémentation de tous les algorithmes

Bresenham complet :

```
void drawLineBresenham(int x0, int y0, int x1, int y1, const Color& color,
Image& img) {
    int dx = abs(x1 - x0);
```


© 2025 Rihen — Tous droits réservés

```
    int dy = abs(y1 - y0);
    int sx = (x0 < x1) ? 1 : -1;
    int sy = (y0 < y1) ? 1 : -1;
    int err = dx - dy;

    while (true) {
        img.setPixel(x0, y0, color);

        if (x0 == x1 && y0 == y1) break;

        int e2 = 2 * err;
        if (e2 > -dy) {
            err -= dy;
            x0 += sx;
        }
        if (e2 < dx) {
            err += dx;
            y0 += sy;
        }
    }
}
```

Scanline polygon fill :

```
void fillPolygon(const vector<Vec2f>& vertices, const Color& color, Image& img)
{
    if (vertices.size() < 3) return;

    // Trouver les bornes y
    int ymin = img.getHeight(), ymax = 0;
    for (const auto& v : vertices) {
        ymin = min(ymin, (int)v.y);
        ymax = max(ymax, (int)v.y);
    }

    // Edge Table
    vector<vector<Edge>> edgeTable(ymax - ymin + 1);

    int n = vertices.size();
    for (int i = 0; i < n; i++) {
        const Vec2f& v1 = vertices[i];
        const Vec2f& v2 = vertices[(i+1) % n];

        if (v1.y == v2.y) continue;

        const Vec2f& bottom = (v1.y < v2.y) ? v1 : v2;
        const Vec2f& top = (v1.y < v2.y) ? v2 : v1;

        float dx = (top.x - bottom.x) / (top.y - bottom.y);

        edgeTable[(int)bottom.y - ymin].push_back({(int)top.y, bottom.x, dx});
```


© 2025 Rihen — Tous droits réservés

```
    }

    // Active Edge Table
    vector<Edge> activeEdges;

    for (int y = ymin; y <= ymax; y++) {
        // Ajouter les nouvelles arêtes
        for (const auto& edge : edgeTable[y - ymin]) {
            activeEdges.push_back(edge);
        }

        // Supprimer les arêtes terminées
        activeEdges.erase(
            remove_if(activeEdges.begin(), activeEdges.end(),
                      [y](const Edge& e) { return e.ymax == y; }),
            activeEdges.end()
        );

        // Trier par x
        sort(activeEdges.begin(), activeEdges.end(),
             [](const Edge& a, const Edge& b) { return a.x < b.x; });

        // Remplir
        for (size_t i = 0; i < activeEdges.size(); i += 2) {
            if (i+1 >= activeEdges.size()) break;

            int x1 = (int)ceil(activeEdges[i].x);
            int x2 = (int)floor(activeEdges[i+1].x);

            for (int x = x1; x <= x2; x++) {
                img.setPixel(x, y, color);
            }
        }

        // Mettre à jour les x
        for (auto& edge : activeEdges) {
            edge.x += edge.dx;
        }
    }
}
```

7.4 Exemple complet : rasterizer d'un fichier OBJ

```
// main.cpp
#include <fstream>
#include <sstream>
#include <vector>

struct Model {
    vector<Vec3f> vertices;
```


© 2025 Rihen — Tous droits réservés

```
    vector<Vec3f> normals;
    vector<Vec2f> texCoords;
    vector<vector<int>> faces;  // indices: 0=vertex, 1=tex, 2=normal
};

Model loadOBJ(const string& filename) {
    Model model;
    ifstream file(filename);
    string line;

    while (getline(file, line)) {
        if (line.empty() || line[0] == '#') continue;

        istringstream iss(line);
        string type;
        iss >> type;

        if (type == "v") {
            Vec3f v;
            iss >> v.x >> v.y >> v.z;
            model.vertices.push_back(v);
        }
        else if (type == "vt") {
            Vec2f vt;
            iss >> vt.x >> vt.y;
            model.texCoords.push_back(vt);
        }
        else if (type == "vn") {
            Vec3f vn;
            iss >> vn.x >> vn.y >> vn.z;
            model.normals.push_back(vn);
        }
        else if (type == "f") {
            vector<int> face;
            string vertex;
            while (iss >> vertex) {
                // Format: v/vt/vn ou v//vn ou v/vt
                replace(vertex.begin(), vertex.end(), '/', ' ');
                istringstream viss(vertex);
                int v, vt, vn;
                viss >> v;
                face.push_back(v);
                if (viss >> vt) face.push_back(vt);
                if (viss >> vn) face.push_back(vn);
            }
            model.faces.push_back(face);
        }
    }

    return model;
}

int main() {
```


© 2025 Rihen — Tous droits réservés

```
    // Créer l'image
    const int width = 800;
    const int height = 600;
    Image img(width, height);
    img.clear(Color(50, 50, 50));  // Gris foncé

    // Z-buffer
    ZBuffer zbuffer(width, height);

    // Charger un modèle
    Model model = loadOBJ("models/cube.obj");

    // Matrices de transformation (MVP)
    Matrix4f modelMatrix = Matrix4f::translate(0, 0, -5);
    Matrix4f viewMatrix = Matrix4f::identity();
    Matrix4f projMatrix = Matrix4f::perspective(45.0f, width/(float)height,
0.1f, 100.0f);
    Matrix4f mvp = projMatrix * viewMatrix * modelMatrix;

    // Rasteriser chaque face
    for (const auto& face : model.faces) {
        // Extraire les 3 sommets (supposons triangles)
        Vertex v0, v1, v2;

        // Transformations
        v0.position = mvp * Vec4f(model.vertices[face[0]-1], 1.0f);
        v1.position = mvp * Vec4f(model.vertices[face[3]-1], 1.0f);
        v2.position = mvp * Vec4f(model.vertices[face[6]-1], 1.0f);

        // Perspective divide
        v0.position = v0.position / v0.position.w;
        v1.position = v1.position / v1.position.w;
        v2.position = v2.position / v2.position.w;

        // Viewport transform
        v0.position.x = (v0.position.x + 1.0f) * width / 2.0f;
        v0.position.y = (1.0f - v0.position.y) * height / 2.0f;
        v1.position.x = (v1.position.x + 1.0f) * width / 2.0f;
        v1.position.y = (1.0f - v1.position.y) * height / 2.0f;
        v2.position.x = (v2.position.x + 1.0f) * width / 2.0f;
        v2.position.y = (1.0f - v2.position.y) * height / 2.0f;

        // Couleurs (RGB selon la position)
        v0.color = Color(255, 0, 0);
        v1.color = Color(0, 255, 0);
        v2.color = Color(0, 0, 255);

        // Rasteriser
        rasterizeTriangle(v0, v1, v2, nullptr, Light(), img, zbuffer);
    }

    // Sauvegarder
    img.savePPM("output.ppm");
```


© 2025 Rihen — Tous droits réservés

```
    return 0;
}
```

Conclusion

Nous avons construit un rasterizer complet depuis zéro, en comprenant :

1. **Pourquoi les approches naïves échouent** : divisions, flottants, trous dans les lignes
2. **Comment Bresenham optimise** : entiers, additions, variable d'erreur
3. **Le remplissage de polygones** : scanline, edge tables, gestion des cas particuliers
4. **La 3D et l'interpolation** : barycentriques, Z-buffer, correction de perspective
5. **Les optimisations** : scanline, edge walking, subpixel precision

Ce cours te donne toutes les bases pour implémenter ton propre moteur de rendu logiciel. Chaque algorithme peut être testé, modifié, optimisé selon tes besoins.

ANNEXE MATHÉMATIQUE : FONDEMENTS DES ALGORITHMES DE RASTERISATION

Introduction

Cette annexe détaille chaque formule mathématique utilisée dans les algorithmes de rasterisation. Pour chaque concept, nous verrons :

- La définition formelle
- L'intuition géométrique
- La démonstration ou justification
- L'application pratique en rasterisation
- L'implémentation C++ optimisée

Annexe A : Géométrie de la droite et pente

A.1 Équation cartésienne d'une droite

Définition formelle :
Dans le plan cartésien, une droite peut être définie par l'équation :
[
y = m x + b
]
où (m) est la **pente** et (b) l'**ordonnée à l'origine**.

Calcul de la pente :
[

© 2025 Rihen — Tous droits réservés

m = \frac{\Delta y}{\Delta x} = \frac{y_1 - y_0}{x_1 - x_0}
]

Intuition géométrique :
La pente représente la variation de (y) pour chaque unité de variation de (x). Si (m = 2), quand (x) augmente de 1, (y) augmente de 2.

Pourquoi cette équation est insuffisante en rasterisation :

1. Division par zéro si (\Delta x = 0) (droite verticale)
2. Utilisation de nombres flottants
3. Accumulation d'erreurs d'arrondi

Forme implicite (plus adaptée) :
[
ax + by + c = 0
]
où (a = dy), (b = -dx), (c = dx \cdot y_0 - dy \cdot x_0)

Avantages de la forme implicite :

- Pas de division
- Fonctionne pour toutes les droites (verticales incluses)
- Permet de tester la position d'un point par rapport à la droite

Implémentation :

```
struct Line {
    float a, b, c;

    // Construire une droite à partir de deux points
    Line(float x0, float y0, float x1, float y1) {
        float dx = x1 - x0;
        float dy = y1 - y0;
        a = dy;
        b = -dx;
        c = dx * y0 - dy * x0;
    }

    // Évaluer la fonction implicite
    float evaluate(float x, float y) const {
        return a * x + b * y + c;
    }

    // Tester le signe (côté de la droite)
    int side(float x, float y) const {
        float val = evaluate(x, y);
        if (val > 0) return 1;   // positif
        if (val < 0) return -1;  // négatif
        return 0;                 // sur la droite
```


© 2025 Rihen — Tous droits réservés

```
    }
};
```

Annexe B : Variable d'erreur de Bresenham

B.1 Définition et intuition

Problème fondamental :
Étant donné une droite de pente (0 < m < 1), on incrémente (x) de 1 à chaque étape. Doit-on incrémenter (y) ou le laisser inchangé ?

Position idéale :
[
y_{\text{ideal}} = m x + b
]

Choix binaire :

- Rester sur (y_{\text{current}}) : erreur = (y_{\text{ideal}} - y_{\text{current}})
- Monter à (y_{\text{current}} + 1) : erreur = ((y_{\text{ideal}} - (y_{\text{current}} + 1)))

On choisit l'option qui minimise l'erreur absolue.

Variable de décision :
[
d = y_{\text{ideal}} - (y_{\text{current}} + 0.5)
]

- Si (d < 0) : point médian plus proche de (y_{\text{current}}) → on reste
- Si (d \ge 0) : point médian plus proche de (y_{\text{current}} + 1) → on monte

Pourquoi 0.5 ? C'est le point médian entre deux pixels. En prenant ce point comme seuil, on minimise l'erreur quadratique moyenne.

B.2 Formulation entière

Étape 1 : Substituer (y_{\text{ideal}} = \frac{\Delta y}{\Delta x} x + b)

[
d = \frac{\Delta y}{\Delta x} x + b - (y_{\text{current}} + 0.5)
]

Étape 2 : Multiplier par (2\Delta x) pour éviter les fractions

[
d' = 2\Delta y \cdot x + 2\Delta x \cdot b - 2\Delta x \cdot y_{\text{current}} - \Delta x
]

Étape 3 : Isoler les termes constants

© 2025 Rihen — Tous droits réservés

[
d' = 2\Delta y \cdot x - 2\Delta x \cdot y_{\text{current}} + C
]
où (C = 2\Delta x \cdot b - \Delta x) est constant.

Étape 4 : Mise à jour incrémentale

Quand on incrémente (x) :
[
d'{\text{new}} = 2\Delta y \cdot (x+1) - 2\Delta x \cdot y + C
]
[
d'{\text{new}} = d'_{\text{old}} + 2\Delta y
]

Quand on incrémente (y) :
[
d'{\text{new}} = 2\Delta y \cdot x - 2\Delta x \cdot (y+1) + C
]
[
d'{\text{new}} = d'_{\text{old}} - 2\Delta x
]

Variable de décision initiale (au premier pixel ((x_0, y_0))) :
[
d'_{\text{start}} = 2\Delta y - \Delta x
]

Algorithme final :

```
int dx = abs(x1 - x0);
int dy = abs(y1 - y0);
int err = dx - dy;  // Version simplifiée (équivalente à d'start)

while (true) {
    setPixel(x, y);
    if (x == x1 && y == y1) break;

    int e2 = 2 * err;
    if (e2 > -dy) {
        err -= dy;
        x += sx;
    }
    if (e2 < dx) {
        err += dx;
        y += sy;
    }
}
```

Pourquoi la version simplifiée fonctionne ?

- On évite de multiplier par 2 à chaque itération
- Les comparaisons sont ajustées en conséquence
- La valeur initiale `dx - dy` est équivalente à (2\Delta y - \Delta x) à un facteur près

---

Annexe C : Coordonnées barycentriques

C.1 Définition formelle

Dans un triangle de sommets (A, B, C), tout point (P) du plan peut s'écrire comme combinaison convexe :
[
P = \alpha A + \beta B + \gamma C
]
avec (\alpha + \beta + \gamma = 1) et (\alpha, \beta, \gamma \in \mathbb{R}).

Pour les points intérieurs au triangle : (\alpha, \beta, \gamma \ge 0).

Interprétation géométrique :

- (\alpha) est proportionnel à l'aire du triangle (PBC)
- (\beta) est proportionnel à l'aire du triangle (APC)
- (\gamma) est proportionnel à l'aire du triangle (ABP)

[
\alpha = \frac{\text{aire}(PBC)}{\text{aire}(ABC)},\quad
\beta = \frac{\text{aire}(APC)}{\text{aire}(ABC)},\quad
\gamma = \frac{\text{aire}(ABP)}{\text{aire}(ABC)}
]

C.2 Calcul pratique via les aires

Aire d'un triangle (déterminant) :
[
\text{aire}(ABC) = \frac{1}{2} \left| (B - A) \times (C - A) \right|
]
En 2D, le produit vectoriel donne un scalaire (le déterminant) :
[
\text{aire}(ABC) = \frac{1}{2} \left| (x_B - x_A)(y_C - y_A) - (y_B - y_A)(x_C - x_A) \right|
]

Calcul des coordonnées barycentriques via les aires signées :

```
float area(const Vec2f& a, const Vec2f& b, const Vec2f& c) {
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
}

Barycentric computeBarycentric(const Vec2f& p, const Vec2f& a,
```


© 2025 Rihen — Tous droits réservés

```
                                const Vec2f& b, const Vec2f& c) {
    float areaABC = area(a, b, c);
    float alpha = area(p, b, c) / areaABC;
    float beta  = area(a, p, c) / areaABC;
    float gamma = area(a, b, p) / areaABC;
    return {alpha, beta, gamma};
}
```

Pourquoi utiliser les aires signées ? Cela évite les valeurs absolues et donne automatiquement le bon signe pour les points extérieurs.

C.3 Méthode matricielle (système linéaire)

Formulation : On cherche (\beta) et (\gamma) tels que :
[
P - A = \beta (B - A) + \gamma (C - A)
]

Soit :
[
\begin{cases}
\beta (x_B - x_A) + \gamma (x_C - x_A) = x_P - x_A \
\beta (y_B - y_A) + \gamma (y_C - y_A) = y_P - y_A
\end{cases}
]

Écriture matricielle :

[
\begin{bmatrix}
x_B - x_A & x_C - x_A \
y_B - y_A & y_C - y_A
\end{bmatrix}
\begin{bmatrix}
\beta \ \gamma
\end{bmatrix}

\begin{bmatrix}
x_P - x_A \ y_P - y_A
\end{bmatrix}
]

C.4 Règle de Cramer

© 2025 Rihen — Tous droits réservés

Règle de Cramer : Pour un système $M \cdot X = V$, chaque inconnue s'obtient par :

$$
\beta = \frac{\det(M_\beta)}{\det(M)},\quad
\gamma = \frac{\det(M_\gamma)}{\det(M)}
$$

où M_β est M avec la première colonne remplacée par V.

Calcul du déterminant :

$$
\det(M) = (x_B - x_A)(y_C - y_A) - (y_B - y_A)(x_C - x_A)
$$

Calcul de β :

$$
\det(M_\beta) = (x_P - x_A)(y_C - y_A) - (y_P - y_A)(x_C - x_A)
$$

Calcul de γ :

$$
\det(M_\gamma) = (x_B - x_A)(y_P - y_A) - (y_B - y_A)(x_P - x_A)
$$

Implementation :

```
Barycentric computeBarycentricCramer(const Vec2f& p, const Vec2f& a,
                                     const Vec2f& b, const Vec2f& c) {
    Vec2f ab = b - a;
    Vec2f ac = c - a;
    Vec2f ap = p - a;

    float det = ab.x * ac.y - ab.y * ac.x;

    // Cas dégénéré (triangle aplati)
    if (fabs(det) < 1e-6) return {0, 0, 0};

    float invDet = 1.0f / det;

    float beta  = (ap.x * ac.y - ap.y * ac.x) * invDet;
    float gamma = (ab.x * ap.y - ab.y * ap.x) * invDet;
    float alpha = 1.0f - beta - gamma;

    return {alpha, beta, gamma};
}
```

Pourquoi Cramer est efficace ici ?

- Le système est 2×2, le calcul du déterminant est direct
- Pas besoin d'inverser la matrice

© 2025 Rihen — Tous droits réservés

- Les formules sont simples et vectorisables

Annexe D : Interpolation avec correction de perspective

D.1 Le problème mathématique

Contexte : Nous avons un triangle 3D avec des attributs (couleur, UV, etc.) variant linéairement en espace 3D :

$$A(X, Y, Z) = \alpha A_0 + \beta A_1 + \gamma A_2$$

où (α, β, γ) sont les coordonnées barycentriques dans l'espace 3D.

Projection perspective :

$$x = \frac{X}{Z}, \quad y = \frac{Y}{Z}$$

Question : Comment interpoler A dans l'espace écran (x, y) ?

Idée fausse : Interpoler A linéairement en (x, y) → résultat incorrect car la perspective introduit une distorsion.

D.2 Démonstration mathématique

Étape 1 : Exprimons A en fonction des coordonnées de l'espace écran.

Les coordonnées barycentriques en espace 3D s'écrivent :

$$\alpha = \frac{\text{aire}(PBC)}{\text{aire}(ABC)}$$

En espace écran, les aires ne sont pas conservées. Mais on peut écrire :

$$\alpha = \frac{\text{aire}(PBC)}{\text{aire}(ABC)} = \frac{\frac{1}{2} | (B-P) \times (C-P) |}{\frac{1}{2} | (B-A) \times (C-A) |}$$

Étape 2 : Utilisons les coordonnées homogènes.

En projection perspective, un point 3D (X, Y, Z) devient en espace écran $(x, y) = (X/Z, Y/Z)$. Mais on utilise plutôt les coordonnées homogènes (x, y, z, w) avec $w = Z$.

Étape 3 : Interpolation linéaire en espace écran.

Soit A un attribut. En espace 3D, A est linéaire en (α, β, γ) :

$$A = \alpha A_0 + \beta A_1 + \gamma A_2$$

© 2025 Rihen — Tous droits réservés

On peut montrer que (A/Z) est linéaire en ((x, y)) :

[
\frac{A}{Z} = \alpha \frac{A_0}{Z_0} + \beta \frac{A_1}{Z_1} + \gamma \frac{A_2}{Z_2}
]

De même, (1/Z) est linéaire :

[
\frac{1}{Z} = \alpha \frac{1}{Z_0} + \beta \frac{1}{Z_1} + \gamma \frac{1}{Z_2}
]

Étape 4 : Reconstruction.

Pour un pixel donné, on calcule :

[
\frac{1}{Z} = \text{lerp}\left(\frac{1}{Z_0}, \frac{1}{Z_1}, \frac{1}{Z_2}\right)
]

[
\frac{A}{Z} = \text{lerp}\left(\frac{A_0}{Z_0}, \frac{A_1}{Z_1}, \frac{A_2}{Z_2}\right)
]

Puis :

[
A = \frac{A/Z}{1/Z}
]

Formule finale :

[
A(x, y) = \frac{\alpha \frac{A_0}{Z_0} + \beta \frac{A_1}{Z_1} + \gamma \frac{A_2}{Z_2}}{\alpha \frac{1}{Z_0} + \beta \frac{1}{Z_1} + \gamma \frac{1}{Z_2}}
]

D.3 Implémentation

```
struct Vertex {
    Vec4f position;  // Position en espace écran (x, y, z, w)
    Vec2f uv;        // Coordonnées de texture
    Color color;     // Couleur
    float z;         // Profondeur (Z)
};

void interpolatePerspective(const Vertex& v0, const Vertex& v1, const Vertex&
v2,
                            float alpha, float beta, float gamma,
                            Vec2f& uv, Color& color, float& z) {
    // Pré-calcul des inverses
    float invZ0 = 1.0f / v0.z;
    float invZ1 = 1.0f / v1.z;
    float invZ2 = 1.0f / v2.z;
```


© 2025 Rihen — Tous droits réservés

```
    // Interpolation de 1/Z
    float invZ = alpha * invZ0 + beta * invZ1 + gamma * invZ2;
    z = 1.0f / invZ;

    // Interpolation de U/Z et V/Z
    float uOverZ = alpha * v0.uv.x * invZ0 +
                   beta  * v1.uv.x * invZ1 +
                   gamma * v2.uv.x * invZ2;
    float vOverZ = alpha * v0.uv.y * invZ0 +
                   beta  * v1.uv.y * invZ1 +
                   gamma * v2.uv.y * invZ2;

    uv.x = uOverZ / invZ;
    uv.y = vOverZ / invZ;

    // Interpolation de la couleur (même principe)
    float rOverZ = alpha * v0.color.r * invZ0 +
                   beta  * v1.color.r * invZ1 +
                   gamma * v2.color.r * invZ2;
    float gOverZ = alpha * v0.color.g * invZ0 +
                   beta  * v1.color.g * invZ1 +
                   gamma * v2.color.g * invZ2;
    float bOverZ = alpha * v0.color.b * invZ0 +
                   beta  * v1.color.b * invZ1 +
                   gamma * v2.color.b * invZ2;

    color.r = rOverZ / invZ;
    color.g = gOverZ / invZ;
    color.b = bOverZ / invZ;
}
```

Annexe E : Test de point dans polygone

E.1 Méthode du ray casting

Principe : Tracer une demi-droite horizontale vers la droite depuis le point (P). Compter le nombre d'intersections avec les arêtes du polygone.

- **Nombre impair** → point à l'intérieur
- **Nombre pair** → point à l'extérieur

Pourquoi ça marche ? Théorème de Jordan : une courbe fermée simple divise le plan en deux régions. Chaque fois qu'on traverse la courbe, on alterne entre intérieur et extérieur.

E.2 Algorithme d'intersection

Pour une arête entre (V_i) et (V_{i+1}), on teste si la demi-droite horizontale intersecte l'arête.

Condition d'intersection :

© 2025 Rihen — Tous droits réservés

1. y est entre y_i et y_{i+1} (strictement)
2. x est à gauche du point d'intersection

Calcul du point d'intersection :

$$x_{\text{intersect}} = x_i + \frac{y - y_i}{y_{i+1} - y_i} \cdot (x_{i+1} - x_i)$$

Condition de gauche : $x < x_{\text{intersect}}$

E.3 Gestion des cas particuliers

Problème : Que faire quand la demi-droite passe exactement par un sommet ?

Solution : Règle de l'extrémité supérieure (top-edge rule) :

- On considère que l'arête inclut son extrémité supérieure mais pas l'inférieure
- En pratique, on teste $(y_i \le y < y_{i+1})$ plutôt que $(y_i \le y \le y_{i+1})$

Implémentation :

```
bool pointInPolygon(const Vec2f& p, const vector<Vec2f>& vertices) {
    bool inside = false;
    int n = vertices.size();

    for (int i = 0, j = n-1; i < n; j = i++) {
        const Vec2f& vi = vertices[i];
        const Vec2f& vj = vertices[j];

        // Vérifier si l'arête intersecte la demi-droite horizontale
        bool intersect = ((vi.y > p.y) != (vj.y > p.y)) &&
                         (p.x < (vj.x - vi.x) * (p.y - vi.y) / (vj.y - vi.y) +
vi.x);

        if (intersect) inside = !inside;
    }

    return inside;
}
```

Pourquoi utiliser `(vi.y > p.y) != (vj.y > p.y)` ?

- Cette condition vérifie que $(p.y)$ est strictement entre $(v_i.y)$ et $(v_j.y)$
- Elle gère automatiquement le cas où $(p.y)$ est exactement égal à un sommet
- Elle est plus rapide que des comparaisons avec `<` et `>=`

Annexe F : Transformations et matrices

F.1 Matrice de projection perspective

© 2025 Rihen — Tous droits réservés

Objectif : Transformer un point 3D ((X, Y, Z)) en coordonnées normalisées ((x, y, z)) où (x, y \in [-1, 1]).

Formule mathématique :

[
x = \frac{X}{Z}, \quad y = \frac{Y}{Z}, \quad z = \frac{\text{near} + \text{far} - 2 \cdot \text{near} \cdot \text{far} / Z}{\text{far} - \text{near}}
]

Forme matricielle :

[
\begin{bmatrix}
\frac{2 \cdot \text{near}}{\text{right} - \text{left}} & 0 & \frac{\text{right} + \text{left}}{\text{right} - \text{left}} & 0 \\
0 & \frac{2 \cdot \text{near}}{\text{top} - \text{bottom}} & \frac{\text{top} + \text{bottom}}{\text{top} - \text{bottom}} & 0 \\
0 & 0 & -\frac{\text{far} + \text{near}}{\text{far} - \text{near}} & -\frac{2 \cdot \text{far} \cdot \text{near}}{\text{far} - \text{near}} \\
0 & 0 & -1 & 0
\end{bmatrix}
]

Pour une caméra symétrique (left = -right, bottom = -top) :

[
\begin{bmatrix}
\frac{2 \cdot \text{near}}{\text{width}} & 0 & 0 & 0 \\
0 & \frac{2 \cdot \text{near}}{\text{height}} & 0 & 0 \\
0 & 0 & -\frac{\text{far} + \text{near}}{\text{far} - \text{near}} & -\frac{2 \cdot \text{far} \cdot \text{near}}{\text{far} - \text{near}} \\
0 & 0 & -1 & 0
\end{bmatrix}
]

Avec le champ de vision (FOV) :

[
\text{height} = 2 \cdot \text{near} \cdot \tan\left(\frac{\text{FOV}}{2}\right)
]
[
\text{width} = \text{height} \cdot \text{aspect ratio}
]

Implémentation :

```
Matrix4f perspective(float fov, float aspect, float near, float far) {
    float tanHalfFov = tan(fov * 0.5f * M_PI / 180.0f);
    float height = 2.0f * near * tanHalfFov;
    float width = height * aspect;
```


© 2025 Rihen — Tous droits réservés

```
    Matrix4f mat;
    mat.data[0][0] = 2.0f * near / width;
    mat.data[1][1] = 2.0f * near / height;
    mat.data[2][2] = -(far + near) / (far - near);
    mat.data[2][3] = -2.0f * far * near / (far - near);
    mat.data[3][2] = -1.0f;
    mat.data[3][3] = 0.0f;

    return mat;
}
```

F.2 Matrice de transformation viewport

Objectif : Transformer les coordonnées normalisées ([-1, 1]) en coordonnées écran ([0, \text{width}] \times [0, \text{height}]).

Formule :

[
x_{\text{screen}} = (x_{\text{ndc}} + 1) \cdot \frac{\text{width}}{2}
]

[
y_{\text{screen}} = (1 - y_{\text{ndc}}) \cdot \frac{\text{height}}{2}
]

Forme matricielle :

[
\begin{bmatrix}
\frac{\text{width}}{2} & 0 & 0 & \frac{\text{width}}{2} \
0 & -\frac{\text{height}}{2} & 0 & \frac{\text{height}}{2} \
0 & 0 & 1 & 0 \
0 & 0 & 0 & 1
\end{bmatrix}
]

Pourquoi (1 - y_{\text{ndc}}) ? Parce que dans l'espace NDC, (y) croît vers le haut, mais dans l'écran, (y) croît vers le bas.

Annexe G : Interpolation linéaire (LERP)

G.1 Définition

LERP (Linear Interpolation) est une méthode pour estimer une valeur entre deux points connus.

Formule scalaire :

[
\text{lerp}(a, b, t) = a + t \cdot (b - a) = (1 - t) \cdot a + t \cdot b
]

où (t \in [0, 1]).

© 2025 Rihen — Tous droits réservés

Propriétés :

- (\text{lerp}(a, b, 0) = a)
- (\text{lerp}(a, b, 1) = b)
- (\text{lerp}(a, b, 0.5) = \frac{a + b}{2}) (milieu)

G.2 Interpolation bilinéaire

Définition : Pour un rectangle, on interpole d'abord horizontalement, puis verticalement.

[
\text{lerp}*{x}(u)* = *\text{lerp}(p*{00}, p_{10}, u)
]
[
\text{lerp}*{x}(d)* = *\text{lerp}(p*{01}, p_{11}, u)
]
[
\text{result} = \text{lerp}(\text{lerp}*{x}(u),* *\text{lerp}*{x}(d), v)
]

Formule directe :
[
P(u, v) = (1-u)(1-v)p_{00} + u(1-v)p_{10} + (1-u)v p_{01} + uv p_{11}
]

Application : Filtrage de texture bilinéaire.

```
Color sampleBilinear(const Texture& tex, float u, float v) {
    int width = tex.getWidth();
    int height = tex.getHeight();

    // Coordonnées du pixel de texture
    float x = u * width - 0.5f;
    float y = v * height - 0.5f;

    int x0 = floor(x);
    int y0 = floor(y);
    int x1 = x0 + 1;
    int y1 = y0 + 1;

    // Facteurs d'interpolation
    float fx = x - x0;
    float fy = y - y0;

    // Échantillonnage des 4 pixels
    Color c00 = tex.getPixel(x0, y0);
    Color c10 = tex.getPixel(x1, y0);
    Color c01 = tex.getPixel(x0, y1);
    Color c11 = tex.getPixel(x1, y1);
```


© 2025 Rihen — Tous droits réservés

```
    // Interpolation horizontale
    Color c0 = c00 * (1 - fx) + c10 * fx;
    Color c1 = c01 * (1 - fx) + c11 * fx;

    // Interpolation verticale
    return c0 * (1 - fy) + c1 * fy;
}
```

Annexe H : Produit vectoriel et déterminant

H.1 Définition en 2D

Pour des vecteurs (\vec{u} = (u_x, u_y)) et (\vec{v} = (v_x, v_y)) en 2D, le produit vectoriel (pseudo-scalaire) est :
[
\vec{u} \times \vec{v} = u_x v_y - u_y v_x
]

Interprétation géométrique :

- C'est l'aire signée du parallélogramme formé par (\vec{u}) et (\vec{v})
- Positif si (\vec{v}) est à gauche de (\vec{u})
- Négatif si (\vec{v}) est à droite de (\vec{u})
- Nul si les vecteurs sont colinéaires

Application : Test d'orientation

```
// Retourne >0 si (a,b,c) tourne à gauche, <0 si à droite, 0 si alignés
float orientation(const Vec2f& a, const Vec2f& b, const Vec2f& c) {
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
}
```

H.2 Produit vectoriel en 3D

Pour (\vec{u} = (u_x, u_y, u_z)) et (\vec{v} = (v_x, v_y, v_z)) :
[
\vec{u} \times \vec{v} =
\begin{pmatrix}
u_y v_z - u_z v_y \
u_z v_x - u_x v_z \
u_x v_y - u_y v_x
\end{pmatrix}
]

Propriétés :

© 2025 Rihen — Tous droits réservés

- (\vec{u} \times \vec{v}) est perpendiculaire à (\vec{u}) et (\vec{v})
- La norme (|\vec{u} \times \vec{v}| = |\vec{u}| |\vec{v}| \sin(\theta))
- Direction donnée par la règle de la main droite

Application : Calcul de normale à un triangle

```
Vec3f computeNormal(const Vec3f& a, const Vec3f& b, const Vec3f& c) {
    Vec3f ab = b - a;
    Vec3f ac = c - a;
    Vec3f normal = ab.cross(ac);
    return normal.normalize();
}
```

Annexe I : Règle de Cramer pour les systèmes 2×2

I.1 Formule générale

Pour un système linéaire :

[
\begin{cases}
a_{11} x + a_{12} y = b_1 \
a_{21} x + a_{22} y = b_2
\end{cases}
]

Déterminant :

[
\Delta = a_{11} a_{22} - a_{12} a_{21}
]

Solutions :

[
x = \frac{b_1 a_{22} - a_{12} b_2}{\Delta}, \quad
y = \frac{a_{11} b_2 - b_1 a_{21}}{\Delta}
]

I.2 Démonstration

À partir du système original, multiplions la première équation par (a_{22}) et la seconde par (a_{12}) :

[
a_{11} a_{22} x + a_{12} a_{22} y = b_1 a_{22}
]

[
a_{12} a_{21} x + a_{12} a_{22} y = b_2 a_{12}
]

© 2025 Rihen — Tous droits réservés

Soustrayons :

[
(a_{11} a_{22} - a_{12} a_{21}) x = b_1 a_{22} - a_{12} b_2
]

D'où (x = \frac{b_1 a_{22} - a_{12} b_2}{\Delta})

De même pour (y).

I.3 Application à l'interpolation barycentrique

Pour le système :

[
\beta (B - A) + \gamma (C - A) = P - A
]

Posons :

[
\vec{u} = B - A = (u_x, u_y), \quad \vec{v} = C - A = (v_x, v_y), \quad \vec{w} = P - A = (w_x, w_y)
]

Système :

[
\begin{cases}
u_x \beta + v_x \gamma = w_x \
u_y \beta + v_y \gamma = w_y
\end{cases}
]

Par Cramer :

[
\Delta = u_x v_y - u_y v_x
]

[
\beta = \frac{w_x v_y - v_x w_y}{\Delta}
]

[
\gamma = \frac{u_x w_y - w_x u_y}{\Delta}
]

Application :

```
float det = u.x * v.y - u.y * v.x;
float beta = (w.x * v.y - v.x * w.y) / det;
float gamma = (u.x * w.y - w.x * u.y) / det;
float alpha = 1.0f - beta - gamma;
```


© 2025 Rihen — Tous droits réservés

Annexe J : Résolution de systèmes linéaires 3×3

J.1 Pour l'éclairage et les transformations

Dans certains contextes (calcul de normales, transformations inverses), on peut avoir besoin de résoudre des systèmes 3×3.

Méthode directe : Formules de Cramer étendues

Pour un système :
[
\begin{cases}
a_{11} x + a_{12} y + a_{13} z = b_1 \
a_{21} x + a_{22} y + a_{23} z = b_2 \
a_{31} x + a_{32} y + a_{33} z = b_3
\end{cases}
]

Déterminant :
[
\Delta = a_{11}(a_{22}a_{33} - a_{23}a_{32}) - a_{12}(a_{21}a_{33} - a_{23}a_{31}) + a_{13}(a_{21}a_{32} - a_{22}a_{31})
]

Solution pour (x) :
[
x = \frac{b_1(a_{22}a_{33} - a_{23}a_{32}) - a_{12}(b_2a_{33} - a_{23}b_3) + a_{13}(b_2a_{32} - a_{22}b_3)}{\Delta}
]

Formules similaires pour (y) et (z).

J.2 Application : Calcul de matrice inverse

Pour inverser une matrice 3×3, on utilise la formule :
[
M^{-1} = \frac{1}{\det(M)} \text{adj}(M)
]
où (\text{adj}(M)) est la matrice adjointe (transposée de la matrice des cofacteurs).

Implémentation :

```
Matrix3f inverse(const Matrix3f& m) {
    float det = m.determinant();
    if (fabs(det) < 1e-6) return Matrix3f::identity(); // Non inversible

    float invDet = 1.0f / det;

    Matrix3f adj;
```


© 2025 Rihen — Tous droits réservés

```
    adj.data[0][0] = (m.data[1][1] * m.data[2][2] - m.data[1][2] * m.data[2]
[1]);
    adj.data[0][1] = (m.data[0][2] * m.data[2][1] - m.data[0][1] * m.data[2]
[2]);
    adj.data[0][2] = (m.data[0][1] * m.data[1][2] - m.data[0][2] * m.data[1]
[1]);
    adj.data[1][0] = (m.data[1][2] * m.data[2][0] - m.data[1][0] * m.data[2]
[2]);
    adj.data[1][1] = (m.data[0][0] * m.data[2][2] - m.data[0][2] * m.data[2]
[0]);
    adj.data[1][2] = (m.data[0][2] * m.data[1][0] - m.data[0][0] * m.data[1]
[2]);
    adj.data[2][0] = (m.data[1][0] * m.data[2][1] - m.data[1][1] * m.data[2]
[0]);
    adj.data[2][1] = (m.data[0][1] * m.data[2][0] - m.data[0][0] * m.data[2]
[1]);
    adj.data[2][2] = (m.data[0][0] * m.data[1][1] - m.data[0][1] * m.data[1]
[0]);

    return adj * invDet;
}
```

Annexe K : Nombres flottants et erreurs d'arrondi

K.1 Problématique

Les calculs en virgule flottante introduisent des erreurs qui peuvent causer :

- Des pixels manquants sur les bords des triangles
- Des artefacts de Z-fighting
- Des tests d'égalité incorrects

K.2 Gestion des tolérances

Règle : Ne jamais tester l'égalité de deux flottants directement.

```
// Mauvais
if (a == b) { ... }

// Bon
const float EPSILON = 1e-6f;
if (fabs(a - b) < EPSILON) { ... }
```

Application aux tests d'appartenance :

```
bool isInsideTriangle(const Barycentric& b) {
    const float EPSILON = 1e-6f;
```


© 2025 Rihen — Tous droits réservés

```
    return b.alpha >= -EPSILON &&
           b.beta  >= -EPSILON &&
           b.gamma >= -EPSILON;
}
```

K.3 Évitement des divisions par zéro

```
float safeDiv(float a, float b) {
    const float EPSILON = 1e-8f;
    if (fabs(b) < EPSILON) return 0.0f;
    return a / b;
}
```

K.4 Utilisation de la virgule fixe

Pour les algorithmes critiques (Bresenham), on utilise des entiers pour éviter les erreurs.

Conversion flottant → fixe :

```
// Format 16.16 (16 bits entiers, 16 bits fractionnaires)
typedef int32_t fixed;

fixed floatToFixed(float f) {
    return (fixed)(f * 65536.0f);
}

float fixedToFloat(fixed f) {
    return f / 65536.0f;
}
```

Annexe L : Optimisations mathématiques

L.1 Éviter les racines carrées

Le calcul de distance euclidienne nécessite une racine carrée, coûteuse.

Comparaison de distances sans racine :

```
// Mauvais
if (distance(p1, p2) < radius) { ... }

// Bon
float dx = p1.x - p2.x;
float dy = p1.y - p2.y;
if (dx*dx + dy*dy < radius*radius) { ... }
```


© 2025 Rihen — Tous droits réservés

L.2 Utilisation de tables précalculées

Pour les fonctions trigonométriques :

```
class TrigTable {
    static const int TABLE_SIZE = 1024;
    float sinTable[TABLE_SIZE];
    float cosTable[TABLE_SIZE];

public:
    TrigTable() {
        for (int i = 0; i < TABLE_SIZE; i++) {
            float angle = 2.0f * M_PI * i / TABLE_SIZE;
            sinTable[i] = sin(angle);
            cosTable[i] = cos(angle);
        }
    }

    float sin(float angle) {
        int idx = (int)(angle * TABLE_SIZE / (2.0f * M_PI)) % TABLE_SIZE;
        return sinTable[idx];
    }
};
```

L.3 Approximations rapides

Inverse rapide (Quake III) :

```
float fastInvSqrt(float x) {
    float xhalf = 0.5f * x;
    int i = *(int*)&x;
    i = 0x5f3759df - (i >> 1);
    x = *(float*)&i;
    x = x * (1.5f - xhalf * x * x);
    return x;
}
```

L.4 Calcul incrémental

Au lieu de recalculer les coordonnées barycentriques pour chaque pixel, on peut les mettre à jour incrémentalement.

Idée : Quand on se déplace d'un pixel à droite :
[
\alpha_{\text{new}} = \alpha_{\text{old}} + \delta\alpha

]
où (\delta\alpha) est constant pour une ligne donnée.

```
// Pré-calcul des incréments
float dAlpha_dx = (b.y - c.y) / area;  // Variation de alpha par pixel x
float dAlpha_dy = (c.x - b.x) / area;  // Variation de alpha par pixel y

// Dans la boucle
alpha += dAlpha_dx;
beta  += dBeta_dx;
gamma += dGamma_dx;
```

Conclusion

Cette annexe couvre les fondements mathématiques essentiels pour la rasterisation :

1. **Géométrie analytique** : équations de droite, formes implicites
2. **Algèbre linéaire** : déterminants, Cramer, matrices de transformation
3. **Interpolation** : LERP, barycentriques, correction de perspective
4. **Optimisations** : calculs incrémentaux, approximations

Chaque formule est présentée avec son contexte d'application et une implémentation pratique. Ces outils mathématiques forment la base de tout rasterizer performant.

© 2025 Rihen — Tous droits réservés